Onyx — Project Overview

Onyx · multi-process OS on Raspberry Pi 4

1. In one sentence

Onyx is a **small multi-process operating system** for the **Raspberry Pi 4** (AArch64), built **on top of Circle** (rsta2/circle), which serves as its HAL and driver stack. It loads **ELF programs from the SD card** and runs them as **applications isolated from one another**, each in its own page table. The whole thing is driven by the **Onyx desktop** — a compositor, a menu bar, a dock, a terminal, a file manager — and over a hundred applications and services (`sdcard/apps`), from an office suite and a web browser to emulators. The same programs also run in a **pocket** and a **console** interface (§3). Our own code is under the MIT licence (LICENSING.md).

It **runs on real Raspberry Pi 4 hardware** (not just in emulation).



The Onyx desktop, a modernised CDE: the menu bar, the agenda on the Voronoi wallpaper, a calculator behind (grey frame), the terminal in front (its frame in the theme's colour), the dock with its Internet drawer open. (The real apps, run on a PC by the desktop simulator.)

2. Philosophy: "own the top, reuse the bottom"

Circle is a *single-process, single-address-space, cooperative* bare-metal framework ("your application **is** the system"). Onyx **keeps the bottom half** of Circle (which is excellent) and **replaces its "operating system" half**:

Layer	Decision	Detail
Reset → EL1, MMU activation, GIC, C++ constructors	**Keep Circle**	Our entry point is `main()`; everything below it is already done.
64 KB page allocator (`palloc`/`pfree`)	**Keep** — it's our frame allocator	`CMemorySystem::Get()`.
GIC-400, timer, mailbox, EMMC+FatFs, USB, Ethernet, framebuffer	**Reuse**	This is the whole point of building on Circle.
`CScheduler`/`CTask` scheduler	**Rewrite the implementation, keep the API**	See kernel internals.
Per-process page tables, context switching, exception vectors, address spaces	**Write from scratch**	Doesn't exist in Circle.

Practical consequence: we **link** the kernel against Circle's already-compiled static libraries (`libcircle.a`, `libfs`, `libusb`, …) rather than copying its sources.

3. Main features

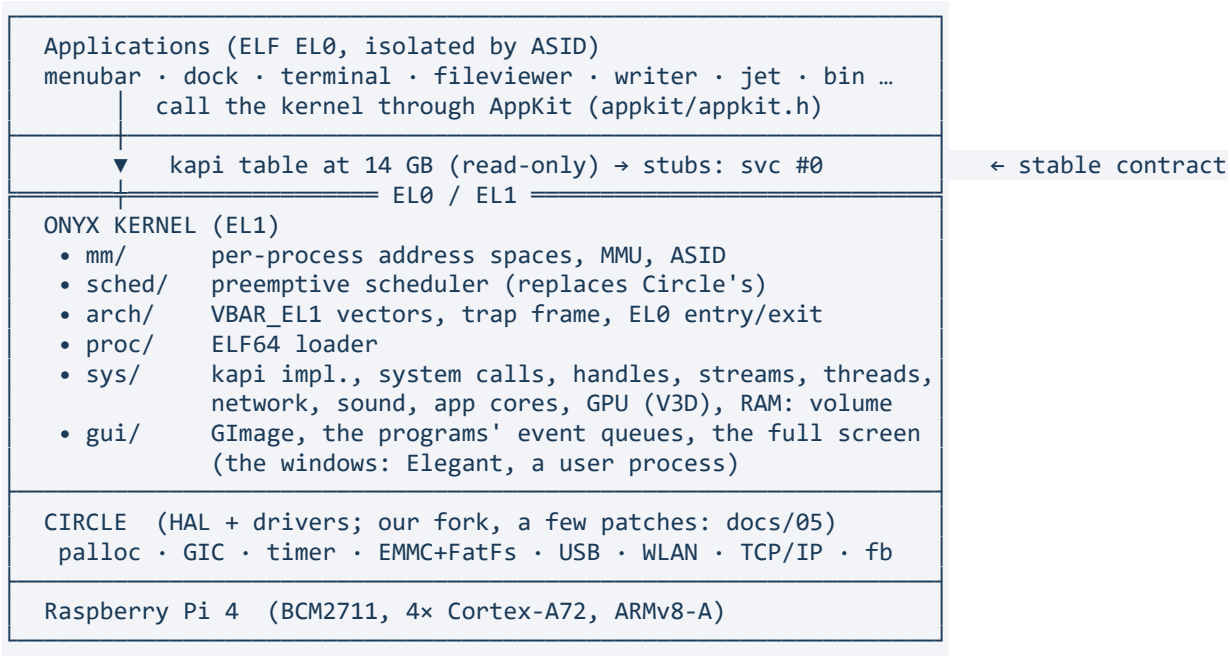
- **Per-process isolation via the MMU.** Each application has its own L2/L3 page table (64 KB granule), tagged by **ASID**. One application cannot see another's memory.
- **Protected apps (EL0).** Applications run at **EL0**, the unprivileged level, and call the kernel by **system calls** (`svc`): they are isolated from one another **and** from the kernel and the hardware. An app that crashes is killed; the system goes on. See §5.
- **Stable fixed-address ABI (`kapi`).** Each application sees a **table of function pointers** at a fixed virtual address (14 GB), mapped read-only; each entry is a small stub that makes the system call. Applications call the kernel through this table → **an application binary keeps working without recompilation** when the kernel changes (current ABI version: **97**). Since AppKit (below) a program no longer reads that table itself: it calls `SD:/lib/appkit.so` by name, and only AppKit follows the kernel.
- **Demand paging** (v75). The heap, the stacks and the mapped regions are given a page only when it is touched; a process that runs out of memory is ended with a notice, the system goes on. There is no swap (the disk is an SD card).
- **Shared images, shared libraries, preloading.** A program's file is read **once**: its code and read-only data are mapped into every process that runs it, only its writable part (`.data`, `.bss`) is copied per process. A **shared library** (`SD:/lib/<name>.so`, v83) is the same object. An image may be **preloaded** — loaded ahead and kept as a template with no process (`SD:/etc/preload.ini`, the Control Panel's Preload applet): starting a large program such as the browser's WebKit is then a mapping and a copy of its writable pages, not a read of the card. (Kernel internals, *Demand paging* and *Shared libraries*.)
- **The kits.** The shared libraries are one **kit per domain**: **AppKit** (the kernel's calls, what makes a program run), **UIKit** (the widgets), **SystemKit** (the system and the other programs), **NetKit**, **FileKit**, **ImageKit**, **AudioKit**, **FontKit**, **PrinterKit**, **GPIOKit**, **GameKit** (the games of the card's

ROMs). A program draws on the kits and never reaches the kernel itself; a kit that changes is replaced on the card, the programs are not rebuilt. (The kits.)

- **Threads, app cores, RAM volume.** An app may run threads (mutexes, events, a futex, "real time" priority), take a whole CPU core for its own code (the emulators, Doom), and keep files in memory on `RAM:`.
- **The GPU.** The VideoCore VI's 3D unit (V3D) is driven by the kernel for the apps: 3D (the N64 and GameCube emulators, the BASIC's 3D), and the compositing of layers (`user/Libs/gpucomp`: Paint's blend modes, Jet Browser).
- **The graphics server is a user process** (2026-10-05): **Elegant** (`SD:/bin/elegant`) has the window manager, the compositor and the routing of the input; the kernel gives it the display, the raw input and shared buffers, starts it at boot and again if it ends (the windows come back with their pixels). The programs reach it through AppKit, unchanged.
- **Full graphical desktop.** 32-bit compositor, window manager, windows with themeable decoration, wallpaper, mouse cursor; the widgets are **UIKit**'s, a shared library (buttons, lists, grids, forms, tabs, toolbars, menus, dialogs…), with themes.
- **Three interfaces, one binary per app.** `shell=` in `SD:/etc/system.ini` (the Control Panel's **Mode** applet; a switch needs no restart) chooses the graphics server: **desktop** (Elegant: windows, the menu bar, the dock), **pocket** (PocketUI: every app full screen, a launcher, a switcher, larger controls, landscape or portrait) or **console** (PocketUI without its bands, the whole screen the app's; its home screen is still to come). Each server loads its own UIKit under the same name, and UIKit's adaptive widgets lay themselves out for the mode: **an app is not rebuilt, and has no code per mode**. (User guide §5, the study.)
- **English and French.** The language is the system's (Language & Region); the apps follow it.
- **Onyx shell.** A menu bar (the active app's menus, the clock, the Wi-Fi and volume menus), a dock (drawers of apps, workspaces), notifications, a shared clipboard with a history, an interactive terminal with **pipes and redirection** (`|`, `>`, `>>`, `<`), a column file browser, a Control Panel, a first-run wizard (Setup), and about a hundred command-line tools in `/bin`.
- **Application catalog** (User guide §12). Office: Letters (word processor: `.docx`, `.odt`, `.rtf`, PDF export), Spreadsheet (`.xlsx`, `.ods`, CSV), Slides (presentations: `.pptx`, `.odp`) — our own programs, written for this system, reading and writing the formats of Word, Excel, PowerPoint and LibreOffice —, Cardfile (a small database), Ledger (accounting), Calendar, PDF Viewer (MuPDF), RTF reader, Archiver. Internet: Jet Browser, Mail, IRC, Courier (HTTP client), Lisa (an AI chat). Media: Paint, Photos, the Media Player (music and video), Screenshot, Koton (a music studio with plugins), FM Tracker. Emulators: Game Boy / Color, GBA, NES, SNES, N64, GameCube, Nintendo DS, with a Game Library. Games (Doom on Freedoom, Tetris, Solitaire, FreeCell, Invaders, Arkanoid…), a BASIC with its editor (QBasic), the Package Manager, a Task Manager, demos.
- **USB input devices.** Keyboard and mouse (HID), USB gamepads and MIDI keyboards, with keyboard layouts loaded from the card (US, UK, DE, FR, BE, ES, IT, Dvorak).
- **Networking (WLAN).** TCP/IP stack + on-board Wi-Fi (BCM4343 / `wpa_supplicant`), on core 3 (`netcore=1`, the shipped card) or core 0; TCP sockets exposed to apps through the ABI, TLS in user space, NTP clock synchronisation, and remote access: a telnet shell, VNC, a window-level remote desktop for Windows (`rdpd` + Onyx Remote), an FTP server.
- **Packages.** Every app is a signed package of the `onyx-packages` repository: `pkg`, the Package Manager and an update daemon keep a card up to date.
- **A modern web browser. Jet Browser** (`jet`), on **WebKit** (the engine of Safari): its own port to Onyx — `http://` and `https://`, JavaScript compiled by JavaScriptCore's JIT, the page assembled

by the Pi's GPU, `<video>` and `<audio>` (VP9, AV1, Opus; YouTube plays), one page per window, three processes (the window, the page, the network): the WebKit port.

4. The architecture at a glance



5. The execution model: apps at EL0

The most structurally defining architectural decision. Since kapi v74 (2026-10-02):

- Applications run at **EL0**, **each in its own page table** tagged by ASID; the kernel's memory, the devices and the other apps' pages are out of their reach (EL1-only mappings).
- They **call the kernel through the `kapi` table** at a fixed address, as before — but in an app that table points at small stubs (`mov x8, #slot; svc #0; ret`): every kapi is a **system call**, its pointers checked, its handles per process. `memcpy` / `memset` and the event pump run in the app itself (no system call). Existing binaries kept working unchanged.
- **A fault in an app kills that app** (a notice on the desktop, a line in `kmsg`), never the machine.
- History: the project first ran apps at EL1 with direct calls ("Option C", `ARCHITECTURE.md` §11–§12: apps isolated from one another but not from the kernel); the move to EL0 was done in steps (EL0-PROTECTED-MODE.md) and the EL1 mode removed.

6. Scheduling: preemptive applications, non-preemptive kernel

A **100 Hz timer tick** gives each task a time slice (20 ms); when it expires, the kernel **preempts the application** and runs the next task. An application stuck in a loop that never yields no longer freezes the system: the cursor, the desktop and the other apps stay responsive, and `taskman` can stop it (the `spin` app tests this).

An IRQ taken while an app runs (at EL0) arrives on that task's own kernel stack, with the app's full register and FP state saved there; when the slice is over the kernel yields right there, like a voluntary switch.

Only **application code** is preempted: an app inside a `kapi_*` call, and the kernel's own threads, are not (the **kernel is non-preemptive**, the classic Unix model); its long operations (file reads and writes, SD-

card waits) yield between pieces. Tasks still also switch **voluntarily** (`yield`, `msleep`, `present`, `wait`, …), and a dynamic priority keeps CPU hogs from starving the tasks that yield (the network, the sound). Details in Kernel internals (§ "Preemptive scheduling").

An application may run **threads** (kapi v67): more tasks in its own address space, with mutexes, events and barriers, and calls posted to its event pump — a thread waits on the network while the window stays responsive. The process ends with its main thread; killing it kills them all. The scheduler's task list has no fixed limit.

7. Target hardware

- **Raspberry Pi 4** (BCM2711, 4× Cortex-A72, ARMv8-A) — primary target.
- **HDMI** output (32 bpp framebuffer, 1024×768 by default, configurable).
- **USB keyboard + mouse**.
- Optional **serial console** (GPIO14/15, 115200 8N1) for the boot log and exception dumps.
- **The four cores, used asymmetrically by design** (not a symmetric SMP kernel): **core 0** runs the kernel, the scheduler, the desktop and every process; **core 1** the sound producer (and, on a hang, the crash record); **cores 2 and 3** are **app cores**, each lent whole to one app for its own code (an emulator's machine, Doom's engine); with `netcore=1` **core 3** runs the network stack instead. Nothing of the kernel, Circle's drivers or FatFs has to be multi-core safe. (docs/02 §12, §14, §11.)
- The RPi 5 is not supported yet (I/O behind the RP1 chip via PCIe — heavier bring-up; the plan: PI5-PORT.md).

8. Repository structure

```
README.md          what Onyx is, how to build / stage / test, where the docs are
ARCHITECTURE.md    the original design + build manifest (historical record)
docs/              THIS documentation (overview, internals, dev/user guides, plans)
kernel/            the Onyx kernel (see docs/02-KERNEL-INTERNALS.md)
user/              the userland: apps (Apps/<name>/), uikit toolkit, libraries, /bin tools
                   (bin/), runtime (crt0.S, user.ld, libc/), the media library (av/)
sdcard/            ready-to-flash files for an RPi 4 (firmware, config, apps, /bin, samples)
sdcard_lite/       the minimal card (the required packages), the rest from the repository
third_party/       vendored libraries (FreeType, MuPDF, FFmpeg, ICU, Skia, curl, …)
tools/             host scripts and tests (packages, screenshots, desktop simulator, …)
pc/                the Windows builds (Koton, the emulators, Onyx Remote)
circle/            Circle, as a git submodule (the fork stephaneweg/circle, branch onyx)
```

> **Legacy docs.** `ARCHITECTURE.md` is the historical design record (its banner says what changed since); where it contradicts this documentation, **`docs/` is authoritative**. Its §11–§12 remain the reference for *why* the first execution model ("Option C", apps at EL1) was chosen; apps run at EL0 since kapi v74 (§5).

9. Further reading

Document	For whom	Contents
02 — Kernel Internals	anyone who wants to understand the kernel	boot, memory/MMU, scheduling, exceptions, ABI, GUI, streams
03 — Developer Guide	anyone who wants to build/compile/extend	toolchain, build, app model, extending the ABI, conventions, debugging

Document	For whom	Contents
04 — User Guide	anyone who wants to use it	SD card, desktop, terminal, files, applications, customization
06 — The kits	anyone writing a program	one kit per domain, how a program uses them (their references: 10 to 19)
05 — Circle Changes	anyone touching `circle/`	the patches of our Circle fork vs upstream `Step51.1.1`
08 — Jet Browser, the WebKit port	the browser	the port of WebKit to Onyx: its status, the patch series, how to build
EL0 protected mode	the execution model	how apps moved to EL0, the design
Licensing	distributors	the licences of everything Onyx contains
Handoff	the next session	where the work stands, the next tasks

10. Quick genesis (milestones)

1. Taking control of `main()` on top of Circle's `sysinit`; serial console.
2. Replacement scheduler ("shadow" header for `CScheduler`).
3. `VBAR_EL1` vectors, trap frame, system-call path.
4. Per-process page tables (`CAddressSpace`), TTBR0/ASID switching.
5. ELF64 loader → process.
6. Framebuffer (`C2DGraphics`) + `GImage` rendering core (ported from the author's FreeBASIC `SimpleOS`).
7. Compositor + window manager; two animated demos running simultaneously.
8. **Switch to Option C** (EL1 apps + direct call) then **fixed-table ABI**.
9. Onyx desktop (panel + applist), stream/stdio subsystem, terminal + `/bin`, file manager.
10. Modal dialogs, themes, app-drawn wallpaper, PID management, keyboard layouts, theme editor (ABI v16).
11. Networking: WLAN bring-up + TCP/IP on the primary core, TCP socket calls (ABI v21), an IRC client, a `/bin/net` tool, and NTP clock sync.
12. The modernised CDE desktop (v64: menu bar, dock, workspaces), the uikit toolkit; the preemptive scheduler; sound on core 1, app cores 2–3 (v51), the GPU (v52); Jet Browser (WebKit); the office suite, the emulators, Koton; threads (v67); the network on core 3; the `RAM:` volume (v71); the packages.
13. **Every app at EL0** (v73–v74, 2026-10): system calls through the same table, per-process handles, every kapi pointer checked; a fault kills the app, not the machine.

14. **Demand paging** (v75), shared images and **shared libraries** (v83), the **kits** — AppKit between the programs and the kernel —, **Elegant**, the graphics server as a user process, several windows per program (v94), English and French.
15. **PocketUI** (v97, 2026-10): the pocket and console interfaces beside the desktop, the same apps in all three.

11. Onyx beside the other Raspberry Pi systems

Written on 2026-10-09. A comparison of designs and of what each system offers — nothing here was measured, and the other systems move: check their own sites.

What Onyx is for. A personal computer's system, simple and whole: what an ordinary user does — the web, mail, letters, spreadsheets and slides in the usual formats, pictures, music and video, PDF, printing, games — is covered by programs written for it, light enough for a Pi 4. It is not meant to run the software of another system, and that sets the comparison.

System	What it is	Beside it, Onyx…
Raspberry Pi OS, Ubuntu… (Linux)	a complete general-purpose system, every Pi, tens of thousands of packages	is small enough to be read whole, comes with its apps, shares and preloads their images, changes of interface without changing of apps; it has far fewer programs, one board, no symmetric multiprocessing
LibreELEC, **RetroPie**, Recalbox…	Linux reduced to one use (a media centre, emulation)	has the media player and the emulators **and** the desktop, the office programs, the browser in one system; theirs emulate more machines and are more tuned
Android / LineageOS	a touch system with its own vast catalogue of apps	has a windowed desktop as well as the pocket mode; no Android app runs on it
RISC OS	Acorn's system: 32-bit, cooperative multitasking, a long-lived catalogue	is 64-bit, preemptive, with every app isolated at EL0 (a faulty app is ended, not the machine); RISC OS has decades of software and of users
Haiku, **Plan 9 / 9front**, the BSDs	other independent systems, whose support of the Pi varies	was written for the Pi 4, with its GPU, its Wi-Fi and a WebKit browser with the JIT and video; they are

System	What it is	Beside it, Onyx…
		self-hosted, the BSDs and Plan 9 multi-user, and all far more tested
Circle, Ultibo	bare-metal frameworks: the application *is* the system	is an operating system on top of one (Circle): processes, protection, a desktop, packages

What Onyx has that is its own.

- **One binary, three interfaces** (desktop, pocket, console), switched without a restart: the toolkit adapts, not the app. On Linux the desktop and the mobile shells are separate projects, and an app follows only when it was written to.
- **Images as templates.** Programs and libraries are read once, shared, and may be preloaded, so that a large program starts by a mapping (the idea of Android's *zygote*, done by the kernel's loader).
- **Whole cores for a program.** An emulator or the sound has a core of its own, with no scheduler in the way; the network has another.
- **A graphics server that can end.** Elegant is a process: the kernel starts it again and the windows come back with their pixels.
- **One hand.** One toolkit, one kit per domain, the same conventions in every app, a system-wide language, signed packages for everything.

What it does not have — by choice.

- **One user.** No accounts, no rights on files (FAT): the simplicity of a personal machine.
- **Cross-developed.** No compiler on the machine; programs are built on a PC.
- **Not binary compatible with anything.** Software is ported or written for Onyx, not installed from another system (the POSIX layer is there for the ports).
- **Asymmetric cores.** Every process runs on core 0; cores 2 and 3 are lent whole (§7). The kernel and the drivers need not be multi-core safe; the price is that several busy processes share one core.

What it does not have — yet, or simply not.

- **Hardware**: the Raspberry Pi 4 and 400 only — the Pi 5 is a plan (PI5-PORT.md); no Bluetooth (a study).
- **Security**: apps are isolated from one another and from the kernel, but there is no sandbox per app (an app reads any file), and the remote accesses (telnet, VNC, FTP) are for a trusted network. The code has not been audited.
- **Beyond the ordinary user**: programming tools on the machine (there is a BASIC), containers, professional software.
- **Maturity**: one developer, few machines tested; the office programs read the usual documents, not every feature of every file.
- **Languages**: English and French; no screen reader.